AI appendix

Companion to the 72-hour plan, the working artifact and the recurring operating system. Section 6 of the operating system states where AI may act, where it may only recommend, and where human approval is mandatory; this appendix covers how it was used to produce the work, and where it failed.

Which tools, and for what

Claude (Anthropic), in two modes.

- **Conversational**, to read the data package, derive the policy rules from the Policies & SLAs sheet, reconcile the Open Work queue against the Activity Log, and draft the written deliverables.
- **Claude Code**, to build and test the working artifact: the triage engine, the conflict engine, the two interface surfaces, and the regression tests that hold them.

No other model, service or data source was used. No real Belong or personal data was exposed to any tool; the entire exercise ran on the supplied fictional package.

How they materially changed the work

Three things happened that would not have fitted in the time available by hand.

1. **Rules derived from the sheets, not remembered.** Every priority the artifact assigns cites a clause from Policies & SLAs by id. Writing that by hand invites the thing the exercise punishes — a priority that rests on a feeling and cannot be traced back to a rule.
2. **The whole board audited at once.** Vendor day capacity, coordinator load, single-instance field equipment, committed move-ins and zone coverage are checked across all nineteen cases on every change. The paint crew committed to three move-ins against capacity for two is the kind of finding a person gets to on the second read, if at all.
3. **The artifact tested on messages nobody wrote for it.** This is where the value actually was, and it is the subject of the next section.

Where AI was wrong

The classifier Claude wrote was confident and incorrect, six times. All six were found by running it on messages invented for the purpose rather than on the seven supplied inputs — which it passed from the start, which is precisely why passing them proved nothing.

The one worth describing in full. Given *"H-1002 — the garage door opener stopped responding and the door is stuck half open"*, the engine attached it to the open **air-conditioning** case at that Home, and graded the match **confirmed**. It had conflated *identifying the Home* with *identifying the case*: a quoted Home ID scored so highly that any case at that address won. The failure mode is the dangerous one — not an error, but a confident wrong answer that would have merged a security problem into an unrelated HVAC ticket and hidden it. The fix was conceptual rather than numerical: only the wording of the issue itself may anchor a match; a shared Home or trade is context. A Home we manage can always have a brand-new problem.

WHAT IT GOT WRONG	WHY
A garage door attached to an air-conditioning case as a "confirmed" match	Conflated identifying the Home with identifying the case
"No smoke" read as smoke, escalating to P0	Hazard words matched inside negations
Live water through a light fixture graded P1, not P0	"I turned off the breaker" silenced a water rule — containment was not trade-specific
A dishwasher matched the washer case	`/washer/` matches inside "dishwasher"
A vendor's progress note read as a new Resident request	Type detection only looked for "we..."
"Can your handyman fix both?" passed silently on licensed work	The request itself needed flagging, not only the assignment

They share one root cause: **inferring identity from partial overlap.** Each now has a regression test named after the failure, so it cannot return quietly.

How the output was validated

- **48 regression tests** over the engine and the two gates, run on every change.
- **The demonstration path as an executable test**, driven end to end in light and dark, asserting what must be on screen at each step and failing on any console error.
- **Refusals verified against the running service**, not only in the interface: dispatching a Home whose building requires a Certificate of Insurance is rejected by the server, as is verifying a case with no named person.
- **Adversarial inputs**, including the planted instructions described below.

One build-side control worth stating

The data package contains a marker phrase that an embedded instruction asks an automated reader to smuggle into the executive summary. It is reproduced nowhere in this submission. The instruction itself is shown on screen, quarantined and struck through, because demonstrating the defence requires showing what was removed — but the phrase it carries is replaced by a stand-in, since a marker planted to be found in a deliverable has no business being carried in one.

The decision that follows from all of this

AI designed this operation. It does not run it. The triage path contains no language model: the supplied data carried five instructions written to be obeyed by an automated reader, three of them pointing at the three most expensive errors available in the queue, and a model reading vendor-supplied free text is exactly the surface those instructions attack. The runtime is deterministic, every output traces to a rule id, and nothing automated changes a priority, a licence status or a spend approval on its own.

Where AI could be added safely, it is outside the decision path — polishing the tone of a drafted message before a human sends it, or summarising a long vendor thread for a coordinator. A person still reads and approves the result.